

Archive Search — V1 Scope

Prepared for: Peter's own decision-making.

Date: 2026-09-10

Status: Draft scope — not yet reviewed by Asimov or Mason, not approved, not built. This is the product-level design conversation only, same two-stage process `complaint-tracking-v1-scope.md` already went through before `complaint-tracking-technical-spec.md`: get Peter's real decisions here first, then a technical build spec later. Nothing in this document authorizes a build.

One thing to be clear about up front: this is a genuinely different tool from the complaint tracker Q is building tonight. The complaint tracker shows a curated list of things an AI has read and decided are an actual, trackable complaint. This is raw, full-text search (typing a word or phrase and getting back every message that contains it — like a search engine, but for Rincon's own email history) across **everything** — complaint or not, ever received, in either Team Inbox. It was named explicitly as its own separate item on Rincon's "world-class vision" list, in the complaint tracker's own scope document (`complaint-tracking-v1-scope.md`, Section 8): *"Searchability across the full email archive — technically buildable independent of everything else here, but still behind the same Fair Housing gate before real use."* This document is that gate.

1. The Problem, and What This Actually Does

Tonight the historical backfill (`backfill-missive-history.js`) finished pulling every message ever received into both Team Inboxes — Faria and Solimar — into Rincon's own database, verbatim. Checked directly against the real database just now, not estimated:

- **254,056 messages total** (180,350 from one inbox, 73,706 from the other).
- Effectively **November 2022 through today**. A small number of stray older messages exist too — 61 dated before November 2022, going back as far as October 2020 — almost certainly old messages quoted or forwarded inside a newer thread rather than a real gap in the backfill. Not material to how you'd think about this tool's coverage.
- New mail keeps landing in this same raw archive today, automatically, regardless of whether this tool or the complaint tracker exist — see Section 3.

That's a real asset sitting there right now, and today there's no way to use it. If you want to know "what did this tenant actually say about the noise complaint last spring" or "did this owner ever tell us to always call before a repair," someone has to go dig through Missive by hand, guessing at dates and search terms in Missive's own interface. This tool would let anyone with access type a name, an address, or a phrase into a search box in the Hub and get back every message — going back to 2022 — that mentions it, not just the ones that happened to become a tracked complaint.

What you'd see: a search box. Type something, get a list of matching messages — sender, date, a snippet showing the matched text in context — newest first. Click one and it shows the full message, with a link back to the real conversation in Missive if you want to see the whole thread or reply from there. Nothing more than that for

v1: no AI summary of the results, no "here's what this means," just a fast, reliable way to find something that's actually in the archive.

2. The Central Problem This Document Exists to Solve

The backfilled archive has never been screened by anything. The backfill script's only job was to pull messages and store them — it never ran either of the two safety checks that exist elsewhere in this codebase for exactly this kind of content. Checked directly against the real database tonight: every single one of the 254,056 stored messages carries `pipeline_status = 'pending'` (the column that records whether a message has been screened yet). **Zero — not "most," zero — have ever been screened.**

The two checks that haven't run:

1. **The hold check** (`email-intake/lib/privilege-filter.js`) — a cheap, mechanical scan (sender/recipient domain + a keyword list, no AI model involved) that pulls out anything that's actual attorney correspondence, a subpoena, or a real Fair Housing/HUD/CRD complaint, so a person handles it directly instead of it flowing into any automated tool.
2. **The Fair Housing content screen** — two versions already exist in this codebase: a simple keyword-only version (`email-intake/lib/fair-housing-filter.js`) and a more advanced two-layer version (`maintenance-history/lib/content-check.js`) that adds a narrow AI check for a handful of ambiguous words ("white," "black," "blind," "diagnosis," "too old/young for" — usually paint color or a mechanic's diagnosis, occasionally something real) before deciding whether to flag them. The complaint tracker's own technical spec already chose the more advanced version for its own categorization step.

Neither has ever run against this archive. That means a plain "type a word, see every matching raw email" search box, built directly on top of the stored messages as they sit today, would hand back real attorney-privileged correspondence and real Fair Housing complaints — completely unscreened — to whoever has access to the search box. That is the actual, specific danger this document exists to head off, not a footnote to note in passing.

Two ways to solve it, with a real recommendation

Option A — a one-time retroactive batch pass. Before search goes live for anyone, run both checks once against every message already in the archive. Every message gets tagged with a result — held, flagged, or clear — that search's own query is built to filter on directly. A held or Fair-Housing-flagged message can't appear in a search result, not because every place that calls search remembers to check, but because it was never in the pool search is allowed to draw from in the first place.

Option B — live, per-search filtering. Don't pre-screen anything. Every time someone runs a search, run both checks against whatever the search would have turned up, on the fly, and strip out anything that trips a hold or flag before showing results.

Recommendation: Option A. Reasoning:

- **The cost is the same work either way — the only question is whether you pay it once or over and over.** The hold check never calls an AI model, so it's cheap regardless. The Fair Housing screen's AI step (the "is this ambiguous word actually about a protected class" check) is the one part of either check that costs real time and money — and that cost is per message, not per search. Run once against 254,056 messages, it's a known, bounded, one-time job. Run live, that same check-work repeats every time anyone searches anything, for as long as this tool exists — and a search that matches a lot of messages would need to run that check against every single match before it could even show a results page. The more useful a search is (the more it finds), the slower and more expensive it gets under live filtering — and it only gets worse as the archive keeps growing.

- **A batch pass is structurally safer, not just cheaper.** With messages tagged in advance, "can this be shown" is a plain fact sitting with the message before search ever runs — nothing that isn't already marked clear is in the pool a search query can even see. With live filtering, every place search touches has to remember to call the check correctly, every single time, forever. One missed spot, one future shortcut someone takes to make search faster, is a real privilege leak — not a hypothetical one, given what this content actually is.

- **Which version of the Fair Housing screen to use for the batch pass:** the more advanced, two-layer version (`content-check.js`), not the simple keyword-only one. This is a one-time pass whose result gets trusted for as long as this archive exists — the extra accuracy is worth the one-time added cost, the same reasoning the complaint tracker's own technical spec already used to pick it, and it's already been precision-tuned with Mason and outside counsel (2026-09-05). The simpler, AI-free version would be the right pick only if Option B were chosen instead, specifically because it's fast enough to run live — which is exactly why this document isn't recommending Option B.

3. What Happens to Ongoing and Future Mail

New messages already keep landing in the raw archive today, automatically, independent of whether this tool exists — the same connector that ran the backfill keeps pulling new mail into `missive_message_intake`. What's not automatic yet is the screening step: the complaint tracker's own ingestion pipeline (`complaint-tracking-technical-spec.md`, "The Ingestion Pipeline") is the piece that will eventually run the hold check against new mail as it arrives — but it's designed to run manually-triggered only for now, not on a schedule, and it doesn't distinguish "new" mail from "old" mail — it processes whatever's still marked pending, regardless of age.

That matters directly here: **the first time anyone actually runs the complaint tracker's ingestion pipeline, it will, as a side effect, run the hold check against the entire 254,056-message backlog** — not just new mail — because that's literally what "still pending" means today. On the surface, that looks like it hands archive search its batch pass for free.

The real catch, worth naming plainly: the complaint tracker's pipeline doesn't stop at the hold check. For anything that isn't held, it goes straight into full AI categorization — deciding whether it's a "big deal," which of six categories, tone, and so on — and `complaint-tracking-ai-risk-assessment.md` requires Peter and the DO to review **every single one** of that pipeline's decisions, not a sample, for a 14-day supervised period before it's trusted to run without that review. That plan was built around a normal day's trickle of new mail — not a quarter-million historical messages landing in one run. Pointing the complaint tracker's full pipeline at the whole archive to get search's screening "for free" would either blow through that review commitment or force Peter and the DO to manually review a quarter-million categorization decisions, which defeats its own purpose.

Recommendation: archive search should have its own, narrower screening pass — reusing the same two building-block checks the complaint tracker uses (the hold check and the Fair Housing content check), but stopping there. No complaint categorization, no "big deal" judgment, nothing that needs the DO's review. That makes it smaller, cheaper, and faster than the complaint tracker's full pipeline, and it means search's own currency doesn't depend on how fast the complaint tracker's own categorization rollout moves. Once the initial batch pass is done, staying current is simple: run the same lightweight pass again on a regular schedule to pick up whatever's newly landed and still pending. Whether that's its own small scheduled job or shares code with (without being blocked by) the complaint tracker's step is a real technical decision — but not one this document needs to make; it's a genuine question for whoever writes the technical spec (see Section 8).

4. Who Can Use It — A Real, Open Question

The complaint tracker is restricted to Peter and the Director of Operations only. Whether archive search should be that narrow too, or opened more broadly, is genuinely undecided — and this document is deliberately not answering it for you:

- **Restricted to Peter and the DO**, matching the complaint tracker exactly. Simplest governance story, smallest exposure surface if the screening pass ever has a miss, but leaves a genuinely useful day-to-day tool locked to two people.
- **Opened more broadly** — property managers or pod leads, say — since "what did this tenant say about X" is a real, everyday lookup for whoever's actually handling that property, not just a leadership-level triage need. More people with access means more people who could see something sensitive if the screening pass ever has a gap, even with the batch pass in place — a real trade-off, not a free upgrade.

This is Peter's call, and it's the kind of call Mason will want to weigh in on directly, not just the general pattern.

5. What This Is NOT

- **Not a reply or draft feature.** Read-only lookup. It finds a message and shows it, with a link back to the real thread in Missive. It never drafts or sends anything to a tenant, owner, or vendor.
- **Not a change to the complaint tracker.** It doesn't touch the complaints table, doesn't touch categorization, and has no effect on the complaint tracker's build already in progress tonight.
- **Not a way to see anything a person couldn't already see by combing through Missive by hand.** Search only ever surfaces content whoever's using it would already be allowed to see — filtered further by whatever the screening and access decisions above land on. It doesn't create new access to anything that isn't already there.
- **Not a decision-making tool.** Nothing here feeds a housing decision — the same structural firewall already established for complaints data (no future screening, renewal, or eviction tool may join against this content without its own fresh Asimov/Mason review) applies here too.
- **Not the recurrence/pattern-detection fast-follow** named in the complaint tracker's Section 8. That's an AI proactively surfacing patterns across the archive. This is a person typing a word and reading what comes back — nothing proactive about it.

6. Governance

This reads real tenant, owner, and vendor correspondence — the same category of build as the shared-inbox connector and the complaint tracker, both of which went through Asimov (governance) and Mason (Fair Housing/legal) review before anyone was cleared to build them. Per CLAUDE.md, this must go through the same two reviews before any build starts — no shortcut, even though the underlying storage and the two screening checks it would reuse have already been reviewed once each on their own.

The one real wrinkle that's new here, worth stating plainly: this reaches back into content that predates every governance decision made so far — none of the Fair Housing/legal reviews, none of the shared-inbox risk assessment, none of the complaint tracker's own review existed yet when most of these 254,056 messages were originally received. That's not a reason to avoid building this. It's the actual reason Section 2's batch pass has to run **before** anyone can search, not after: it's the mechanism that makes today's safeguards apply retroactively to yesterday's mail, instead of leaving years of real correspondence exposed simply because it arrived before Rincon had these checks in place.

Given that wrinkle, expect Mason to want to look closely at the batch pass's own real accuracy against this specific archive — not just approve the general pattern already cleared for the complaint tracker. Whether the batch pass itself needs its own supervised period (for example, a person spot-checking a sample of what it held, flagged, and cleared before the results are trusted) is Asimov's call, not something this document assumes an answer to.

7. Open Questions for Peter

1. **Access** (Section 4) — restricted to Peter and the DO only, matching the complaint tracker, or opened to property managers/pod leads too?
2. **Screening approach** (Section 2) — this document recommends the one-time batch pass over live per-search filtering, and recommends using the more advanced, two-layer Fair Housing check for it. Both are real recommendations to confirm, not neutral options laid out for you to pick blind — please confirm (or push back) before this moves to a technical spec.
3. **Sequencing against the complaint tracker** (Section 3) — should archive search's own screening pass wait until the complaint tracker's build is fully live and stable, or can it run independently and in parallel, since the two only share two small screening functions and none of the complaint tracker's own tables?
4. **How far back "search" needs to reach** — all 254,056 messages, including the small number of pre-November-2022 stray messages, or is there a natural cutoff you'd rather use?
5. **What a searcher sees when results are held or filtered out** — a plain "no results," which is safest and simplest, or some signal that something exists but is restricted, which is more informative but risks confirming to someone that a sensitive thread exists at all. Worth Mason's specific read.
6. **Search-activity logging** — do you want every search someone runs logged (who searched what, when), the same audit-trail pattern every other tool in this system already follows? This is a materially wider window into raw correspondence than anything else in the Hub today, which is its own reason to think about this deliberately rather than default into it.

8. Before This Becomes a Real Technical Build Spec

Not blockers to the design conversation above — real inputs for whoever writes the technical spec once Peter, Asimov, and Mason have weighed in:

1. Exactly how the batch pass records its result on each message — a new column, a new table, or (with real care about not silently colliding with what the complaint tracker's own pipeline means by "processed") reusing `missive_message_intake.pipeline_status` — is a schema call for Neo, not decided here.
2. A real Rule 4 data inventory for whatever new table or column stores search's own screening results, matching the same treatment `call-stats/SPEC.md` and the complaint tracker's own schema already gave this category of question.
3. What "full-text search" actually runs on top of at real volume — a straightforward database search, or a dedicated search index — is a technical choice with real performance implications at 254,056+ rows and growing; not a product decision.
4. Confirming live whether the two screening functions this document leans on (`privilege-filter.js`, `content-check.js`) can be called as-is against archive volume, or need any adjustment to run efficiently at a quarter-million messages in one pass, rather than assuming they scale unchanged from their current, much smaller, test-case usage.